

Data Integrity Audit Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

The data integrity audit identified 26 findings across 8 categories, including 8 Critical, 8 High, 6 Medium, and 4 Low severity issues. The most critical findings are data leakage bugs: school admins and teachers can see data from other schools and teachers because queries are not properly scoped. The teacher dashboard counts unique exam IDs instead of students, analytics date range filters are computed but never applied, and there are severe N+1 query patterns in the reports service that will cause timeouts at scale. Additionally, multiple list queries are unbounded (no pagination), which will cause memory issues and slow load times with growing data.

Category	Critical	High	Medium	Low
N+1 Queries	4	0	0	0
Pagination	4	2	0	0
Ordering	1	1	0	0
Error Handling	0	4	1	0
Loading States	0	0	3	0
Optimistic Updates	0	0	0	2
Null Handling	1	1	3	1
Data Consistency	3	1	0	0

Critical Findings

DC2: School Admin Sees ALL Pending Submissions Across Platform

The `getSchoolAdminStats` function counts `pendingSubmissions` from `exam_sessions` with status: "submitted" but does NOT filter by `school_id`. This means a school admin sees ALL pending submissions across the entire platform, not just their school. This is a data leakage bug that exposes cross-school data. Fix: Add `.eq("school_id", schoolId)` or join through exams to filter by school.

DC3: Teacher Sees ALL Pending Grading Across Platform

The `getTeacherStats` function counts `pendingGrading` from `exam_sessions` with status: "submitted" but does NOT filter by the teacher's exams. A teacher sees ALL pending submissions, not just for their own exams. Fix: Add filter by teacher's exam IDs: `.in("exam_id", teacherExamIds)`.

DC4: Reports School Exam Sessions Not Filtered by School

In the school reports loop, `sessionsResult` queries `exam_sessions` but does not filter by school or by the specific school's exams. Every school gets the same session data. Fix: Filter sessions by the school's exam IDs or add `.eq("school_id", school.id)`.

O2/NH1: Teacher Dashboard Counts Exams Instead of Students

The `uniqueStudents.add(session.exam_id)` adds exam IDs instead of student IDs. The `uniqueStudents` set is used to count students, but it is actually counting unique exam IDs. This is a data correctness bug. Fix: Change to `uniqueStudents.add(session.student_id)` and add `student_id` to the query select.

N1-1 through N1-4: Severe N+1 Query Patterns

The analytics service loops over each school (up to 10) making 2 queries per school (20 sequential queries). The reports service loops over each school (up to 50) making 5 parallel queries per school (250 queries total). The teacher and student reports have similar N+1 patterns. These will cause timeouts at scale. Fix: Batch-fetch all data with `school_id IN (...)` or `created_by IN (...)`, then aggregate in-memory with a `Map`.

O1: Analytics Date Range Filter Never Applied

The `dateFilter` variable is computed from the `dateRange` parameter but never used in the query. All analytics data is fetched unfiltered regardless of the selected date range. Fix: Add `.gte("created_at", dateFilter)` to the sessions query when `dateFilter` is not null.

P1-P4: Unbounded Queries on Core Entities

The schools, students, teachers, and parents services all use `select("*")` with no `.limit()` or `.range()`. These queries fetch ALL records with ALL columns. For a school with thousands of students or a platform with hundreds of schools, this will be very slow and memory-intensive. Fix: Add `.limit()` and `.range()` with cursor/offset pagination, and select only needed columns.

High Severity Findings

E1: Search Service Silent Errors

All 8 Supabase queries in the search service destructure only `{data}` and ignore the error return. If any query fails, it silently returns empty results. No error is logged or propagated. Fix: Destructure `{data, error}` for each query and handle errors appropriately.

E5: No Rollback on Failed Optimistic Updates

The notifications page performs optimistic updates before the server action completes, but never rolls back if the server action fails. The server actions return `{error}` but the client code ignores it. Fix: Check the return value and revert the optimistic update on failure.

DC1: Inconsistent Revenue Calculation

Dashboard and reports compute revenue differently (different filtering by status, different date ranges). Fix: Use a shared utility for revenue calculation, or use Supabase RPC for consistency.

Priority Remediation Order

Priority	Action	Impact
P0	Fix data leakage: scope queries by school/teacher	Data security: school admins see cross-school data

Priority	Action	Impact
P0	Fix teacher dashboard: count students not exams	Data correctness: dashboard shows wrong numbers
P0	Fix N+1 queries in reports service	Performance: will timeout at scale
P0	Fix analytics date range filter	Data correctness: all analytics unfiltered
P0	Add pagination to all list queries	Performance: unbounded queries will fail at scale
P1	Add error handling to search service	Reliability: silent failures mask issues
P1	Add rollback to optimistic updates	Data consistency: UI shows incorrect state on failure
P2	Consolidate revenue calculation	Consistency: different numbers in different views